Cloud Infrastructure Cost Review

AWS Small-Scale Environment

Purpose

This document reviews a hypothetical small-scale production environment hosted on AWS and identifies practical steps to reduce monthly cloud expenditure. The analysis focuses on five established cost management techniques: instance rightsizing, committed-use pricing, scheduled scaling, storage tiering, and volume type migration. Each recommendation is intended to be actionable without disrupting the running workload.

Environment Overview

The environment under review supports a small B2B web application serving roughly 500 active users per day. Traffic follows a predictable pattern — heavier during business hours, with very little activity overnight or on weekends. The current monthly bill sits at approximately $558, broken down as follows:

Resource	Current Configuration	Est. Cost/Month
App Server (EC2)	m5.xlarge — 4 vCPU, 16 GB RAM, On-Demand	$140
Batch Worker (EC2)	m5.large — 2 vCPU, 8 GB RAM, On-Demand	$70
Managed Database	RDS db.m5.large, Multi-AZ, 500 GB gp2 EBS	$280
Object Storage	S3 Standard — 2 TB, no lifecycle rules	$46
Load Balancer	Application Load Balancer (ALB), low traffic	$22
Total		**~$558**

Note: Costs above are estimates based on standard AWS us-east-1 On-Demand pricing as of early 2026. Actual figures will vary by region and any existing discount agreements.

Recommendations

The five recommendations below address the most straightforward savings opportunities in this environment. They are ordered roughly by ease of implementation.

1. Rightsize the Application Server

The app server is currently running on an m5.xlarge, but CloudWatch metrics show average CPU utilization around 18% and memory usage around 35%. This is a classic over-provisioning pattern — the instance was likely sized for an anticipated peak that hasn't materialized. Downsizing to an m5.large would cut that line item in half and still leave headroom for traffic spikes.

How to do it:

– Pull 14-30 days of CPU and memory metrics from CloudWatch to confirm the utilization pattern.

– Spin up an m5.large in a staging environment, run a basic load test, and confirm response times are acceptable.

– Schedule a brief maintenance window, swap the production instance, and monitor for 48 hours.

Estimated savings: ~$70/month (~$840/year).

2. Switch to Compute Savings Plans

Both EC2 instances are currently billed at On-Demand rates, which is fine for unpredictable workloads but expensive for anything that runs consistently. A 1-year Compute Savings Plan typically yields a 30-40% discount on compute spend with no operational changes required — the instances just keep running as-is. The 'Compute' flavor (as opposed to an EC2 Instance Savings Plan) is preferable here because it applies across instance families, so it remains valid even if the instance type is changed later.

How to do it:

– Complete the rightsizing change first, so the commitment reflects the correct baseline.

– Open AWS Cost Explorer and navigate to Savings Plans > Recommendations. The tool will suggest a commitment amount based on recent usage.

– Purchase a 1-year Compute Savings Plan with no-upfront or partial-upfront payment. No instance changes are needed.

Estimated savings: ~$42-56/month (~$500-670/year) after rightsizing is applied.

3. Stop the Batch Worker Outside Working Hours

The batch worker processes nightly jobs that wrap up by 11 PM, but the instance continues running through the night until business hours resume. That's roughly 8-9 hours of idle compute time each day. Using a scheduled Auto Scaling action — or simply a Lambda function triggered by EventBridge — to stop the instance at 11 PM and start it again at 8 AM would eliminate that waste without any changes to the jobs themselves.

How to do it:

– Confirm the batch job finish time is consistent (add a CloudWatch alarm if it occasionally runs long).

– Create two EventBridge Scheduler rules: one to stop the instance at 11:15 PM, one to start it at 7:45 AM.

– Test for a week in a non-critical period before treating it as standard operating procedure.

Estimated savings: ~$26/month (~$312/year), assuming ~15 active hours per day instead of 24.

4. Apply S3 Storage Lifecycle Rules

The S3 bucket holds 2 TB on Standard storage tier at $0.023 per GB. A review of object access logs shows that a significant portion — primarily older log exports and database backups — hasn't been touched in over 90 days. S3 Infrequent Access (IA) costs about half as much as Standard for storage, and Glacier Instant Retrieval is cheaper still for anything that just needs to be retained rather than actively used. Lifecycle rules handle the transitions automatically, requiring no ongoing manual work.

How to do it:

– Enable S3 Storage Class Analysis on the bucket and let it run for 30 days to build an accurate access frequency picture.

– Set a lifecycle rule to move objects to Standard-IA at 30 days and to Glacier Instant Retrieval at 90 days.

– Add an expiration rule for log files older than 365 days, assuming data retention policy allows it.

Estimated savings: ~$12/month (~$144/year) on an estimated 1.2 TB transition to cheaper tiers.

5. Migrate RDS Storage from gp2 to gp3

The RDS instance is using gp2 EBS volumes, which was the standard choice for several years but has since been superseded by gp3. The gp3 volume type is cheaper per GB ($0.092 vs $0.115) and actually delivers better baseline I/O performance — 3,000 IOPS included at no extra cost, versus gp2's burstable model. For a database at this scale, gp3 is a straightforward improvement on both dimensions. The change can be made through the AWS Console with no downtime.

How to do it:

– In the RDS Console, select the instance and choose Modify.

– Under Storage, change the volume type from gp2 to gp3. Leave IOPS at the 3,000 default unless monitoring shows higher sustained demand.

– Apply the change immediately (or during the next maintenance window) — no application restart is required.

Estimated savings: ~$11.50/month (~$138/year) on 500 GB of RDS storage.

Summary

Recommendation	Monthly Savings	Annual Savings	Complexity
1. Rightsize app server	~$70	~$840	Low
2. Compute Savings Plans	~$42–$56	~$500–$670	Low
3. Stop batch worker overnight	~$26	~$312	Medium
4. S3 lifecycle rules	~$12	~$144	Low
5. RDS gp2 to gp3	~$11.50	~$138	Low
Total	**~$161–$175**	**~$1,934–$2,104**	

Taken together, these five changes could reduce the monthly bill from around $558 to roughly $383-$397 — a reduction of about 29-31%. Four of the five recommendations involve no code changes and minimal risk. The scheduled stop for the batch worker is the only item that requires a short testing period before it can be relied upon. A reasonable approach would be to implement recommendations 1, 4, and 5 first (they're independent of each other), then proceed with the Savings Plan purchase once rightsizing is stable, and tackle the scheduling work in the following sprint.
